21 WORKER THREADS — DEEP DIVE

theory/20-single-thread-vs-multithreading.txt already introduced worker_threads as ONE of three ways to get real parallelism in Node. This file zooms in on that one technique specifically — what it is, when it actually helps, and — just as important — when it does NOT.

Think of a Worker Thread as a SECOND JavaScript "worker" that can run heavy calculations without blocking your main NestJS thread. It has its own V8 instance and its own event loop, running truly in parallel with the main thread.

A SIMPLE EXAMPLE — SUMMING 1 TO 1 BILLION

Imagine your NestJS API receives:

```
GET /calculate
```

and needs to calculate:

```
1 + 2 + 3 + 4 + ... + 1,000,000,000
```

That's a CPU-heavy, purely synchronous calculation — no database, no network, just the CPU grinding through a billion additions:

```
let result = 0;
for (let i = 0; i < 1_000_000_000; i++) {
  result += i;
}
```

WITHOUT A WORKER — THE MAIN THREAD IS BUSY CALCULATING

```
NestJS Main Thread
   |
   |-- Request 1: GET /calculate
   |     Calculate 1 -> 1 billion
   |         |
   |      BLOCKED (this can take real seconds of 100% CPU)
   |
   |-- Request 2 has to WAIT, no matter what it is — even a
   |   totally unrelated GET /task or GET /user request queues up
   |   behind the calculation, because there's only ONE thread and
   |   it's fully occupied.
```

This is exactly the slowFibonacci() problem from theory/20-single-thread-vs-multithreading.txt — same failure mode, different calculation.

WITH A WORKER THREAD — THE CALCULATION MOVES OFF THE MAIN THREAD

You send the calculation to another thread and get notified when it's done — the main thread is free the entire time it's running:

```
NestJS Main JavaScript Thread
   |
   |  "Calculate this" ----------------> Worker Thread
   |                                          |
   |                                    1 + 2 + 3 + ...
   |                                          |
   |  <------------------------------- Result
   |
   (main thread was free to handle OTHER requests this whole time)
```

Under load, with several requests arriving while the calculation runs:

```
NestJS Main Thread                    Worker Thread
   |                                       |
Request A ------------------------->  (not involved)
   |
Request B ------------------------->  (not involved)
   |
Request C  -- "calculate" -------->   Heavy calculation running
   |                                       |
Request D ------------------------->  (not involved)
   |                                       |
   |   <----------- Result ---------- Calculation done
   |
Requests A, B, D were all served NORMALLY while C's heavy
calculation ran in parallel on the worker thread.
```

RUNNABLE CODE — SAME SHAPE AS theory/20's FIBONACCI EXAMPLE

```ts
// sum.worker.ts — runs ON THE WORKER THREAD
import { parentPort, workerData } from 'worker_threads';

let result = 0;
for (let i = 0; i < workerData.limit; i++) {
  result += i;
}
parentPort?.postMessage(result);

// calculate.service.ts — runs ON THE MAIN THREAD
import { Worker } from 'worker_threads';
import { Injectable } from '@nestjs/common';

@Injectable()
export class CalculateService {
  sumInWorker(limit: number): Promise<number> {
    return new Promise((resolve, reject) => {
      const worker = new Worker('./dist/sum.worker.js', {
        workerData: { limit },
      });
      worker.once('message', resolve);   // worker sent the result back
```

```
      worker.once('error', reject);
      worker.once('exit', (code) => {
        if (code !== 0) reject(new Error(`Worker stopped with code ${code}`));
      });
    });
  }
}

// calculate.controller.ts
@Controller()
export class CalculateController {
  constructor(private readonly calculateService: CalculateService) {}

  @Get('calculate')
  async calculate() {
    return { result: await this.calculateService.sumInWorker(1_000_000_000) };
  }
}
```

The controller method is `async` and `await`s a Promise — but note carefully WHAT that Promise is waiting on: not I/O, but a MESSAGE from another thread that's doing real parallel CPU work. That's the mechanism that keeps the main thread free.

WHAT KIND OF WORK ACTUALLY BELONGS IN A WORKER

Anything CPU-heavy and synchronous — work that would otherwise occupy the main thread for a noticeable stretch of time:

- Processing a huge array / large in-memory dataset
- Image processing
- Video processing
- Encryption / hashing (especially slow ones like bcrypt at high cost factors, or heavy custom crypto)
- Mathematical / numerical calculations
- Parsing a huge file
- Data transformation over large payloads
- Machine-learning inference/training work
- Generating large reports (PDF generation, big spreadsheet exports)

The common thread (no pun intended): these all consume the CPU for a while with no natural "waiting" point — nothing to `await`, because nothing external is being waited on. A worker lets that happen on a DIFFERENT JavaScript thread, so the main NestJS thread isn't stuck doing it.

THE ONE IMPORTANT THING TO REMEMBER — WORKERS ARE NOT FOR I/O

A worker thread is NOT automatically useful for normal database/API calls. For something like:

```
const users = await database.find();
```

you normally do NOT need a worker. Node's asynchronous, non-blocking I/O already lets the main thread go do other work while waiting for the database response — that's the entire point of `await` on I/O (see the findOne() example in theory/20-single-thread-vs-multithreading.txt). Wrapping a database call in a worker thread adds

real overhead (spinning up a thread, message-passing serialization) for ZERO benefit, since the main thread was never actually blocked by that `await` in the first place.

Worker Threads	= move CPU-HEAVY, SYNCHRONOUS JavaScript work off the main thread.
NOT	= "make every API request faster" or "a general performance button." Using one for an I/O-bound call (a DB query, an external API fetch) is pure overhead with no upside.

★ RULE OF THUMB

Before reaching for a worker thread, ask: "if I removed all my `await`s, would this code still take a long time, purely from the CPU grinding through it?" If yes (a tight loop, heavy math, image/file processing) — worker thread. If no (the delay is really just "waiting on the network/DB to respond") — plain `async` / `await` already handles it, and a worker would only add overhead.

See also: **theory/20-single-thread-vs-multithreading.txt**